

File permissions in Linux

Project description

The report consists of the work of a security professional in a large organization, where they need to manage file permissions through the Linux operating system environment. The task is to assign user permission types in a directory called “projects” based on the organization’s needs.

Check file and directory details

The first process is to check the permissions set for all files and subdirectories included in the projects directory. To do this, I used the `ls -lr` command. After running it, the following output, shown in the image below, will appear.

```
researcher2@12dd8ce717b0:~$ ls -l
total 4
drwxr-xr-x 3 researcher2 research_team 4096 Feb 24 21:49 projects
researcher2@12dd8ce717b0:~$ cd projects
researcher2@12dd8ce717b0:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Feb 24 21:49 .
drwxr-xr-x 3 researcher2 research_team 4096 Feb 24 22:21 ..
-rw--w---- 1 researcher2 research_team  46 Feb 24 21:49 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Feb 24 21:49 drafts
-rw-rw-rw- 1 researcher2 research_team  46 Feb 24 21:49 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Feb 24 21:49 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Feb 24 21:49 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Feb 24 21:49 project_t.txt
researcher2@12dd8ce717b0:~/projects$
```

Figure 01: File and directory details

Describe the permissions string

Let’s break this last output into parts.

First, the `ls -l` command, highlighted in red, was used to identify all the files and subdirectories in the current directory. As shown in the next image, this directory contains only one subdirectory called “projects”, highlighted in blue, which is the directory we want to work on.

```
researcher2@12dd8ce717b0:~$ ls -l Command  
total 4  
drwxr-xr-x 3 researcher2 research_team 4096 Feb 24 21:49 projects Directory
```

Figure 02: Permission string commands

With that in mind, I changed to the projects directory using the `cd` command, highlighted in red, and then type the `ls -la` command, highlighted in blue. This command works similarly to `ls -l`, but the letter “a” at the end tells the operating system to display not only the regular files and directories, but also the hidden ones that are not usually shown.

```
researcher2@12dd8ce717b0:~$ cd projects Changes directory  
researcher2@12dd8ce717b0:~/projects$ ls -la Returns all contents  
total 32  
drwxr-xr-x 3 researcher2 research_team 4096 Feb 24 21:49 .  
drwxr-xr-x 3 researcher2 research_team 4096 Feb 24 22:21 ..  
-rw--w--- 1 researcher2 research_team 46 Feb 24 21:49 .project_x.txt  
drwx--x--- 2 researcher2 research_team 4096 Feb 24 21:49 drafts  
-rw-rw-rw- 1 researcher2 research_team 46 Feb 24 21:49 project_k.txt  
-rw-r----- 1 researcher2 research_team 46 Feb 24 21:49 project_m.txt  
-rw-rw-r-- 1 researcher2 research_team 46 Feb 24 21:49 project_r.txt  
-rw-rw-r-- 1 researcher2 research_team 46 Feb 24 21:49 project_t.txt  
researcher2@12dd8ce717b0:~/projects$
```

Figure 03: Permission string output

Now, let’s move on to the next step: understanding file and directory permissions.

Take a look at each line. Every line corresponds to a file or directory, and the name of the file or directory appears at the end of the line. When it is a directory, its name is shown in blue, while files are shown in white.

At the beginning of each line, we can see the permissions of the file or directory. Let’s take the permissions of the `.project_k.txt` file as an example.

Its permissions are represented by a ten-character string: `-rw-rw-rw-`. The first character informs whether the content is a file or a directory. If it is a file, the symbol is `-`; if it is a directory, the symbol is `d`.

The remaining nine characters represent the permissions for each type of user in the system. In Linux, there are three types of users: the user, the group, and others. Each one can have three types of permissions: read, write, and execute.

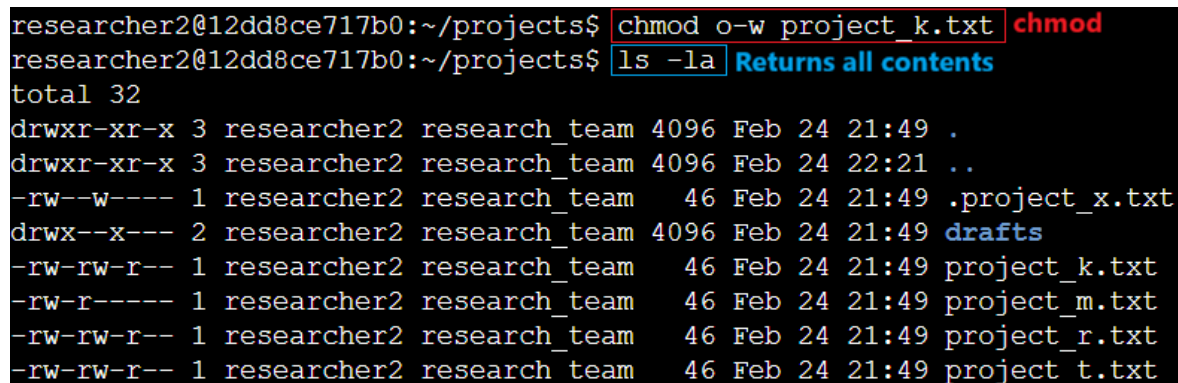
Looking at the nine characters after the first one, `rw-rw-rw-`, the first three characters correspond to the user permissions, the next three correspond to the group permissions, and the final three correspond to the permissions for others. By analyzing them, we can see that all user types have permission to read and write, but not to execute.

Keep in mind that, when dealing with file permissions, read allows the user to view the file content, write allows modifications, and execute allows the file to be executed if it is executable. On the other hand, for directories, read allows the user to view the directory contents, write allows the creation of new files or directories, and execute allows the user to enter the directory and access its contents.

With all that being said, let's move to the next step of this report, which is changing file permissions.

Change file permissions

The organization does not allow other users to have write access to any files. To change permissions, I used the `chmod` command.



```
researcher2@12dd8ce717b0:~/projects$ chmod o-w project_k.txt chmod
researcher2@12dd8ce717b0:~/projects$ ls -la Returns all contents
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Feb 24 21:49 .
drwxr-xr-x 3 researcher2 research_team 4096 Feb 24 22:21 ..
-rw--w---- 1 researcher2 research_team  46 Feb 24 21:49 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Feb 24 21:49 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Feb 24 21:49 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Feb 24 21:49 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Feb 24 21:49 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Feb 24 21:49 project_t.txt
```

Figure 04: Commands and output of changing the file permissions

The `chmod` command, highlighted in red, was used with two arguments: `o-w` and `project_k.txt`.

Let's start with the second argument, `project_k.txt`. This is the name of the file whose permissions we want to change. Based on image 2, we know that the only file where other users have write permission is `project_k.txt`, so the modifications only need to be applied to this file.

The first argument contains the permission modification. The `o` character stands for others, the minus sign `-` removes a permission, and the `w` represents write permission. In other words, the

command tells the operating system to remove write access from other users for the `project_k.txt` file.

After these modifications, we type the `ls -la` command again to check the permissions after the changes and verify whether everything was configured correctly.

Change file permissions on a hidden file

The next change is on the hidden file `.project_x.txt`. For this file, the organization requested that only the user and the group should be able to read it. This can easily be done with a single command, as shown below.

```
researcher2@12dd8ce717b0:~/projects$ chmod u=r,g=r .project_x.txt
researcher2@12dd8ce717b0:~/projects$ ls -la Command
total 32 Returns all contents
drwxr-xr-x 3 researcher2 research_team 4096 Feb 24 21:49 .
drwxr-xr-x 3 researcher2 research_team 4096 Feb 24 22:21 ..
-r--r----- 1 researcher2 research_team  46 Feb 24 21:49 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Feb 24 21:49 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Feb 24 21:49 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Feb 24 21:49 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Feb 24 21:49 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Feb 24 21:49 project_t.txt
```

Figure 05: Changing file permissions on a hidden file

The `chmod` command allows us to make multiple permission changes at once by separating them with a comma (,). In the command above, highlighted in red, we already know that the file being modified is `.project_x.txt`.

The equals sign (=) in the following arguments, `u=r`, sets the permissions exactly as specified. This means that the user will have only read permission, and any write or execute permissions, if they existed, will be removed. The same process was applied to the group with `g=r`.

Then, the `ls -la` command was used again to verify and confirm the changes.

Change directory permissions

Finally, the last change is related to the `drafts` directory. This directory must only be accessed by the `researcher2` user. The `researcher2` user is the owner of the `drafts` directory, as shown on the second line of the `ls -la` output. This means that neither group users nor other users should have access to the `drafts` directory.

By analyzing the last `ls -la` output, we can see that the group users still have execute permission for the `drafts` directory, so this permission must be removed.

```
researcher2@12dd8ce717b0:~/projects$ chmod g-x drafts Command
researcher2@12dd8ce717b0:~/projects$ ls -la Returns all contents
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Feb 24 21:49 .
drwxr-xr-x 3 researcher2 research_team 4096 Feb 24 22:21 ..
-r--r----- 1 researcher2 research_team  46 Feb 24 21:49 .project_x.txt
drwx----- 2 researcher2 research_team 4096 Feb 24 21:49 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Feb 24 21:49 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Feb 24 21:49 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Feb 24 21:49 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Feb 24 21:49 project_t.txt
```

Figure 06: Changing directory permissions

After this change, only the user has permission to read, write, and execute within the `drafts` directory.

Summary

All tasks were completed according to the organization's requirements. The team needed to manage user permissions while balancing authentication and authorization. These changes strengthen the system and make it safer and more reliable.